

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

Д. О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 4

ОРГАНИЗАЦИЯ ПЕРЕДАЧИ УПРАВЛЯЮЩИХ КОМАНД НА
УДАЛЕННОЕ ОБОРУДОВАНИЕ

по курсу:

ИНТЕРНЕТ ВЕЩЕЙ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4326

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт-Петербург 4326

1 Цель работы

Цель работы: приобретение навыков организации передачи управляющих команд на удаленное оборудование.

2 Задание

Организовать передачу управляющих команд из интерфейса на оборудование для всех вещей. Передачу управляющих команд можно реализовать как при помощи нескольких запросов, по одному для каждой вещи, так и одним запросом, который будет передавать все команды, а их обработкой будет заниматься отдельная функция на сервере. В рамках данной лабораторной работы необходимо реализовать передачу именно управляющих команд, переделывать запросы из прошлой лабораторной не нужно.

Вариант задания: «Умная теплица».

3 Ход выполнения

Для разделения сценариев чтения состояния и приема команд у базового класса устройства и наследников был расширен метод connect: при отсутствии аргумента выполняется опрос текущих параметров объекта, при передаче объекта HTTP-запроса параметры из строки запроса считываются и применяются к полям вещи. Для датчиков были предусмотрены включение и выключение по префиксу имени в запросе и калибровка по числовому полю; для исполнительных механизмов — задание мощности, переключение питания и передача текстовой команды; для контроллера — смена режима и сброс тревог; для подсистемы хранения данных — прием текстовой управляющей команды. Структура классов датчика и исполнительного устройства с разбором запроса приведена на рисунках 1–2.

```

35 class Sensor(Device):
36 >     def __init__(...
43 >     ):...
49
50 >     def measure(self):...
55
56 >     def calibrate(self, value: float):...
61
62     def _apply_control(self, request: Request) → None:
63         p = self.control_prefix
64         power = request.args.get(f"{p}_power", "").strip().lower()
65         if power in ("on", "1", "true"):
66             self.turn_on()
67         elif power in ("off", "0", "false"):
68             self.turn_off()
69
70         raw_cal = request.args.get(f"{p}_calibrate", "").strip()
71         if raw_cal:
72             try:
73                 self.calibrate(float(raw_cal.replace(",", ".")))
74             except ValueError:
75                 print(
76                     f"[LOG] Пропуск калибровки {self.name}: некорректное число {raw_cal!r}"
77                 )
78
79     def connect(
80         self, request: Optional[Request] = None
81     ) → Dict[str, Union[int, float, str, bool]]:
82         if request is not None:
83             self._apply_control(request)
84             print(
85                 f"[LOG] Управляющая команда для датчика {self.name}: "
86                 f"значение {self.value} {self.unit}, статус включения {self.status}"
87             )
88         else:
89             print(
90                 f"[LOG] Опрос датчика {self.name}: текущее значение {self.value} {self.unit}"
91             )
92         return {
93             "id": self.id,
94             "name": self.name,
95             "type": self.type,
96             "unit": self.unit,
97             "value": self.value,
98             "status": self.status,
99         }

```

Рисунок 1 — Обновленный класс Sensor

```

102 class Actuator(Device):
103 >     def __init__(...
105 > ):...
110
111 >     def apply_action(self, action: str):...
115
116 >     def set_power(self, level: float):...
121
122     def _apply_control(self, request: Request) → None:
123         p = self.control_prefix
124         power_kw = request.args.get(f"{p}_power_cmd", "").strip()
125         if power_kw:
126             try:
127                 self.set_power(float(power_kw.replace(",", ".")))
128             except ValueError:
129                 print(
130                     f"[LOG] Пропуск установки мощности {self.name}: некорректное число"
131                 )
132
133         action = request.args.get(f"{p}_action", "").strip()
134         if action:
135             self.apply_action(action)
136
137         st = request.args.get(f"{p}_power_state", "").strip().lower()
138         if st in ("on", "1", "true"):
139             self.turn_on()
140         elif st in ("off", "0", "false"):
141             self.turn_off()
142
143     def connect(
144         self, request: Optional[Request] = None
145     ) → Dict[str, Union[int, float, str, bool]]:
146         if request is not None:
147             self._apply_control(request)
148             print(
149                 f"[LOG] Управление {self.name}: мощность {self.power} Вт, статус {self.status}"
150             )
151         else:
152             print(
153                 f"[LOG] Опрос исполнительного устройства {self.name}: {self.power} Вт, статус: {self.status}"
154             )
155         return {
156             "id": self.id,
157             "name": self.name,
158             "type": self.type,
159             "power": self.power,
160             "status": self.status,
161         }
162

```

Рисунок 2 — Обновленный класс Actuator

Класс главного контроллера и класс базы данных были доработаны по той же схеме: вызов connect с запросом изменяет только явно заданные поля, а при опросе без запроса возвращается актуальное состояние (теперь без эмуляции). Их реализация показана на рисунках 3–4.

```

164 class MainControlUnit:
165 >     def __init__(self):...
169
170 >     def process_data(self, sensor_data: Dict[int, float]):...
172
173 >     def send_command(self, actuator: Actuator, command: str):...
178
179 >     def check_alerts(self):...
185
186     def _apply_control(self, request: Request) → None:
187         mode = request.args.get("controller_mode", "").strip().lower()
188         if mode in ("auto", "manual"):
189             self.mode = mode
190             print(f"[LOG] Контроллер: режим установлен в {self.mode}")
191         clear = request.args.get("controller_clear_alerts", "").strip().lower()
192         if clear in ("1", "true", "yes", "on"):
193             self.alerts = []
194             print("[LOG] Контроллер: список тревог очищен")
195
196     def connect(
197         self, request: Optional[Request] = None
198     ) → Dict[str, Union[str, int, List[str]]]:
199         if request is not None:
200             self._apply_control(request)
201             print(
202                 f"[LOG] Подключение к контроллеру выполнено, режим: {self.mode}, количество тревог: {len(self.alerts)}"
203             )
204             return {
205                 "mode": self.mode,
206                 "alerts_count": len(self.alerts),
207                 "alerts": list[str](self.alerts),
208             }

```

Рисунок 3 — Обновленный класс MainControlUnit

```

211 class Database:
212 >     def __init__(self, connection_string: str, storage_path: str):...
216
217 >     def save_reading(self, sensor_id: int, value: float, timestamp: datetime):...
222
223 >     def save_event(self, device_id: int, action: str, timestamp: datetime):...
228
229 >     def get_history(...
231 >     ):...
237
238     def _apply_control(self, request: Request) → None:
239         cmd = request.args.get("database_command", "").strip()
240         if cmd:
241             print(f"[LOG] БД: принята управляющая команда {cmd!r}")
242
243     def connect(self, request: Optional[Request] = None) → Dict[str, Union[str, int]]:
244         if request is not None:
245             self._apply_control(request)
246             data = {
247                 "connection": "ok",
248                 "storage_path": self.storage_path,
249                 "records_today": self.records_today,
250             }
251             print(
252                 f"[LOG] Подключение к БД выполнено, записей за сегодня: {data['records_today']}, путь: {data['storage_path']}"
253             )
254             return data

```

Рисунок 4 — Обновленный класс Database

В модуле маршрутизации были сохранены отдельные конечные точки вида «/connect/temperature» и аналогичные для остальных вещей, возвращающие JSON при вызове connect без аргументов. Для передачи команд был введен единый маршрут «GET /connect/command», в обработчике которого в объект ответа объединялись результаты вызова «connect(request)»

для каждой вещи. На рисунке 5 представлен метод `connect_command`.

```
64 @app.route("/connect/command", methods=["GET"])
65 def connect_command():
66     """Применить управляющие параметры ко всем вещам."""
67     return jsonify(
68         {
69             "temperature": temperature_sensor.connect(request),
70             "humidity": humidity_sensor.connect(request),
71             "soil": soil_sensor.connect(request),
72             "fan": fan.connect(request),
73             "pump": pump.connect(request),
74             "controller": controller.connect(request),
75             "database": database.connect(request),
76         }
77     )
```

Рисунок 5 — Конечная точка для отправки команд устройствам

Пользовательский интерфейс был дополнен блоком управления: для каждой вещи добавлены поля ввода и переключатели, сгруппированные по карточкам, а также кнопка отправки и многострочное поле, куда выводится JSON-ответ сервера после успешного запроса. Дополненная страница приведена на рисунке 6.

The screenshot displays a web interface divided into two main sections: 'Агроном (оператор)' on the left and 'Управление оборудованием' on the right.

Агроном (оператор) Panel:

- Датчик температуры:** 20 °C
- Датчик влажности воздуха:** 44 %
- Датчик влажности почвы:** 43 %
- Вентилятор:** 110 Вт; статус: true
- Насос:** 70 Вт; статус: true
- Контроллер:** Режим: auto, Тревоги: 0
- База данных:** Соединение: ok, Записей за сегодня: 0

Управление оборудованием Panel:

- Датчик температуры:** Питание: Вкл, Калибровка, °C: 20
- Датчик влажности воздуха:** Питание: Выкл, Калибровка, %: 44
- Датчик влажности почвы:** Питание: Вкл, Калибровка, %: 43
- Вентилятор:** Мощность, Вт: 110, Питание: Вкл, Действие (лог): замедлить
- Насос:** Мощность, Вт: 70, Питание: Вкл, Действие (лог): полив
- Контроллер:** Режим: Авто, Сбросить тревоги: ☒
- База данных:** Команда (лог): резервное копирование
- Отправить команды:** Button
- Ответ сервера:**

```
{
  "controller": {
    "alerts": [],
    "alerts_count": 0,
    "mode": "auto"
  },
  "database": {
    "connection": "ok",
    "records_today": 0,
    "storage_path": "/data"
  },
  "fan": {
    "id": 101,
    "name": "Вентилятор",
    "power": 110,
    "status": true,
    "type": "fan"
  },
  "humidity": {
    "id": 2,
    "name": "Датчик влажности воздуха",
    "status": false,
    "type": "humidity",
    "unit": "%",
    "value": 44
  },
  "pump": {
    "id": 102,
    "name": "Насос полива",
    "power": 70
  }
}
```

Рисунок 6 — Обновленный пользовательский интерфейс

На клиенте по-прежнему с заданным интервалом вызываются AJAX-функции к прежним URL опроса. Для управления реализованы сбор параметров формы в объект и отправка метода «GET /connect/command». Код

отправки управляющих команд показан на рисунке 7.

```
108 function collectCommandPayload() {
109     return {
110         temperature_power: $("#temperature_power").val(),
111         temperature_calibrate: $("#temperature_calibrate").val(),
112         humidity_power: $("#humidity_power").val(),
113         humidity_calibrate: $("#humidity_calibrate").val(),
114         soil_power: $("#soil_power").val(),
115         soil_calibrate: $("#soil_calibrate").val(),
116         fan_power_cmd: $("#fan_power_cmd").val(),
117         fan_power_state: $("#fan_power_state").val(),
118         fan_action: $("#fan_action").val(),
119         pump_power_cmd: $("#pump_power_cmd").val(),
120         pump_power_state: $("#pump_power_state").val(),
121         pump_action: $("#pump_action").val(),
122         controller_mode: $("#controller_mode_cmd").val(),
123         controller_clear_alerts: ($("#controller_clear_alerts").is(":checked")
124             ? "1"
125             : ""),
126         database_command: $("#database_command").val(),
127     };
128 }
129
130 function sendCommands() {
131     $.ajax({
132         type: "GET",
133         url: "/connect/command",
134         dataType: "json",
135         contentType: "application/json",
136         data: collectCommandPayload(),
137         success: function (response) {
138             $("#command_result").val(JSON.stringify(response, null, 2));
139         },
140     });
141 }
142
143 $("#send_commands").on("click", sendCommands);
```

Рисунок 7 — JS-функции отправки команд на сервер

На рисунке 8 приведен пример логирования после отправки команд, сформированных на рисунке 6 и отправленных на сервер.

```
[LOG] Датчик температуры (ID: 1) включён
[LOG] Датчик Датчик температуры (ID: 1) откалиброван на значение 20.0 °C
[LOG] Управляющая команда для датчика Датчик температуры: значение 20.0 °C, статус включения True
[LOG] Датчик влажности воздуха (ID: 2) выключен
[LOG] Датчик Датчик влажности воздуха (ID: 2) откалиброван на значение 44.0 %
[LOG] Управляющая команда для датчика Датчик влажности воздуха: значение 44.0 %, статус включения False
[LOG] Датчик влажности почвы (ID: 3) включён
[LOG] Датчик Датчик влажности почвы (ID: 3) откалиброван на значение 43.0 %
[LOG] Управляющая команда для датчика Датчик влажности почвы: значение 43.0 %, статус включения True
[LOG] Мощность устройства Вентилятор (ID: 101) установлена на 110.0 Вт
[LOG] Исполнительное устройство Вентилятор (ID: 101) выполняет действие: замедлить
[LOG] Вентилятор (ID: 101) включён
[LOG] Управление Вентилятор: мощность 110.0 Вт, статус True
[LOG] Мощность устройства Насос полива (ID: 102) установлена на 70.0 Вт
[LOG] Исполнительное устройство Насос полива (ID: 102) выполняет действие: полив
[LOG] Насос полива (ID: 102) включён
[LOG] Управление Насос полива: мощность 70.0 Вт, статус True
[LOG] Контроллер: режим установлен в auto
[LOG] Контроллер: список тревог очищен
```

Рисунок 8 — Результаты передачи команд

В ходе развития системы методы эмуляции случайных значений были удалены: при мониторинге отображаются сохраненные в объектах величины (в т.ч. начальные и измененные командами), что приближает поведение к

передаче реальных команд на фиксированное состояние оборудования.

4 Выводы

В ходе работы была реализована схема передачи управляющих команд с веб-страницы на серверное приложение и далее на логические модели устройств «умной теплицы». Показано, что поток данных при управлении устроен так же, как при опросе мониторинга: клиент формирует HTTP-запрос, сервер обрабатывает его и возвращает ответ в формате JSON, который используется для обновления интерфейса. Для разделения сценариев чтения и записи в методах устройств был использован необязательный параметр запроса: при отсутствии запроса возвращается текущее состояние объекта, при передаче объекта запроса параметры из строки запроса интерпретируются как команды и отражаются в полях соответствующей вещи.

Был выбран вариант одного объединенного маршрута для отправки параметров управления сразу ко всем элементам схемы, что упрощает клиентский код и наглядно демонстрирует централизованную обработку команд на сервере. Интерфейс дополнен формой ввода и полем вывода JSON-ответа, что позволяет проверить корректность разбора параметров и формирования ответа без дополнительных инструментов.

В целом работа показала, как приемы передачи данных мониторинга переносятся на сценарий управления: меняется семантика параметров и обработка на стороне «оборудования», а общий шаблон «запрос, разбор, обновление состояния, ответ» сохраняется.

ПРИЛОЖЕНИЕ

Исходный код

Исходный код приложения также доступен по ссылке:

<https://github.com/grigoriptomczuk/smart-greenhouse>.

Листинг 1 — app.py

```
import logging

from flask import Flask, jsonify, render_template, request

from devices import Actuator, Database, MainControlUnit, Sensor

app = Flask(__name__)

temperature_sensor = Sensor(1, "Датчик температуры", "temperature", "°C",
"temperature")
humidity_sensor = Sensor(2, "Датчик влажности воздуха", "humidity", "%", "humidity")
soil_sensor = Sensor(3, "Датчик влажности почвы", "soil_moisture", "%", "soil")
fan = Actuator(101, "Вентилятор", "fan", 120.0)
pump = Actuator(102, "Насос полива", "pump", 80.0)
controller = MainControlUnit()
database = Database("sqlite:///greenhouse.db", "./data")

temperature_sensor.turn_on()
humidity_sensor.turn_on()
soil_sensor.turn_on()
fan.turn_on()
pump.turn_on()

@app.route("/")
def index():
    return render_template("dashboard.html")

@app.route("/connect/temperature")
def connect_temperature():
    return jsonify(temperature_sensor.connect())

@app.route("/connect/humidity")
def connect_humidity():
    return jsonify(humidity_sensor.connect())

@app.route("/connect/soil")
def connect_soil():
    return jsonify(soil_sensor.connect())

@app.route("/connect/fan")
def connect_fan():
    return jsonify(fan.connect())

@app.route("/connect/pump")
def connect_pump():
```

```

        return jsonify(pump.connect())

@app.route("/connect/controller")
def connect_controller():
    return jsonify(controller.connect())

@app.route("/connect/database")
def connect_database():
    return jsonify(database.connect())

@app.route("/connect/command", methods=["GET"])
def connect_command():
    """Применить управляющие параметры ко всем вещам."""
    return jsonify(
        {
            "temperature": temperature_sensor.connect(request),
            "humidity": humidity_sensor.connect(request),
            "soil": soil_sensor.connect(request),
            "fan": fan.connect(request),
            "pump": pump.connect(request),
            "controller": controller.connect(request),
            "database": database.connect(request),
        }
    )

if __name__ == "__main__":
    logging.getLogger("werkzeug").setLevel(logging.ERROR)
    app.run(debug=True)

```

Листинг 2 — devices.py

```

from abc import ABC, abstractmethod
from datetime import datetime
from typing import Dict, List, Optional, Union

from werkzeug.wrappers import Request

class Device(ABC):
    def __init__(self, device_id: int, name: str, status: bool = False):
        self.id = device_id
        self.name = name
        self.status = status

    def turn_on(self):
        self.status = True
        print(f"[LOG] {self.name} (ID: {self.id}) включён")

    def turn_off(self):
        self.status = False
        print(f"[LOG] {self.name} (ID: {self.id}) выключен")

    def get_status(self) -> bool:
        print(f"[LOG] Запрос статуса {self.name} (ID: {self.id}) -> {self.status}")
        return self.status

    @abstractmethod
    def connect(

```

```

        self, request: Optional[Request] = None
    ) -> Dict[str, Union[int, float, str, bool]]:
        """Мониторинг (request is None) или применение команд (передан request)."""

class Sensor(Device):
    def __init__(
        self,
        device_id: int,
        name: str,
        sensor_type: str,
        unit: str,
        control_prefix: str,
    ):
        super().__init__(device_id, name)
        self.type = sensor_type
        self.unit = unit
        self.control_prefix = control_prefix
        self.value: float = 0.0

    def measure(self):
        self.value = 22.5
        print(
            f"[LOG] Датчик {self.name} (ID: {self.id}) измерил: {self.value} {self.unit}"
        )

    def calibrate(self, value: float):
        self.value = value
        print(
            f"[LOG] Датчик {self.name} (ID: {self.id}) откалиброван на значение {value} {self.unit}"
        )

    def _apply_control(self, request: Request) -> None:
        p = self.control_prefix
        power = request.args.get(f"{p}_power", "").strip().lower()
        if power in ("on", "1", "true"):
            self.turn_on()
        elif power in ("off", "0", "false"):
            self.turn_off()

        raw_cal = request.args.get(f"{p}_calibrate", "").strip()
        if raw_cal:
            try:
                self.calibrate(float(raw_cal.replace(",", ".")))
            except ValueError:
                print(
                    f"[LOG] Пропуск калибровки {self.name}: некорректное число {raw_cal!r}"
                )

    def connect(
        self, request: Optional[Request] = None
    ) -> Dict[str, Union[int, float, str, bool]]:
        if request is not None:
            self._apply_control(request)
            print(
                f"[LOG] Управляющая команда для датчика {self.name}: "
                f"значение {self.value} {self.unit}, статус включения {self.status}"
            )
        else:

```

```

        print(
            f"[LOG] Опрос датчика {self.name}: текущее значение {self.value}
{self.unit}"
        )
        return {
            "id": self.id,
            "name": self.name,
            "type": self.type,
            "unit": self.unit,
            "value": self.value,
            "status": self.status,
        }

class Actuator(Device):
    def __init__(
        self, device_id: int, name: str, actuator_type: str, power: float = 0.0
    ):
        super().__init__(device_id, name)
        self.type = actuator_type
        self.power = power
        self.control_prefix = actuator_type

    def apply_action(self, action: str):
        print(
            f"[LOG] Исполнительное устройство {self.name} (ID: {self.id}) выполняет
действие: {action}"
        )

    def set_power(self, level: float):
        self.power = level
        print(
            f"[LOG] Мощность устройства {self.name} (ID: {self.id}) установлена на
{level} Вт"
        )

    def _apply_control(self, request: Request) -> None:
        p = self.control_prefix
        power_kw = request.args.get(f"{p}_power_cmd", "").strip()
        if power_kw:
            try:
                self.set_power(float(power_kw.replace(",", ".")))
            except ValueError:
                print(
                    f"[LOG] Пропуск установки мощности {self.name}: некорректное
число"
                )

        action = request.args.get(f"{p}_action", "").strip()
        if action:
            self.apply_action(action)

        st = request.args.get(f"{p}_power_state", "").strip().lower()
        if st in ("on", "1", "true"):
            self.turn_on()
        elif st in ("off", "0", "false"):
            self.turn_off()

    def connect(
        self, request: Optional[Request] = None
    ) -> Dict[str, Union[int, float, str, bool]]:
        if request is not None:

```

```

        self._apply_control(request)
        print(
            f"[LOG] Управление {self.name}: мощность {self.power} Вт, статус
{self.status}"
        )
    else:
        print(
            f"[LOG] Опрос исполнительного устройства {self.name}: {self.power}
Вт, статус: {self.status}"
        )
    return {
        "id": self.id,
        "name": self.name,
        "type": self.type,
        "power": self.power,
        "status": self.status,
    }

class MainControlUnit:
    def __init__(self):
        self.mode = "auto"
        self.schedule: Dict[str, str] = {}
        self.alerts: List[str] = []

    def process_data(self, sensor_data: Dict[int, float]):
        print(f"[LOG] Контроллер обрабатывает данные с датчиков: {sensor_data}")

    def send_command(self, actuator: Actuator, command: str):
        print(
            f"[LOG] Контроллер отправляет команду '{command}' устройству
{actuator.name} (ID: {actuator.id})"
        )
        actuator.apply_action(command)

    def check_alerts(self):
        print("[LOG] Контроллер проверяет наличие тревог...")
        if self.alerts:
            print(f"[LOG] Активные тревоги: {self.alerts}")
        else:
            print("[LOG] Тревог нет")

    def _apply_control(self, request: Request) -> None:
        mode = request.args.get("controller_mode", "").strip().lower()
        if mode in ("auto", "manual"):
            self.mode = mode
            print(f"[LOG] Контроллер: режим установлен в {self.mode}")
        clear = request.args.get("controller_clear_alerts", "").strip().lower()
        if clear in ("1", "true", "yes", "on"):
            self.alerts = []
            print("[LOG] Контроллер: список тревог очищен")

    def connect(
        self, request: Optional[Request] = None
    ) -> Dict[str, Union[str, int, List[str]]]:
        if request is not None:
            self._apply_control(request)
        print(
            f"[LOG] Подключение к контроллеру выполнено, режим: {self.mode},
количество тревог: {len(self.alerts)}"
        )
        return {

```

```

        "mode": self.mode,
        "alerts_count": len(self.alerts),
        "alerts": list(self.alerts),
    }

class Database:
    def __init__(self, connection_string: str, storage_path: str):
        self.connection_string = connection_string
        self.storage_path = storage_path
        self.records_today: int = 0

    def save_reading(self, sensor_id: int, value: float, timestamp: datetime):
        self.records_today += 1
        print(
            f"[LOG] БД: сохранено показание датчика {sensor_id} = {value} в {timestamp}"
        )

    def save_event(self, device_id: int, action: str, timestamp: datetime):
        self.records_today += 1
        print(
            f"[LOG] БД: сохранено событие устройства {device_id}: {action} в {timestamp}"
        )

    def get_history(
        self, start: datetime, end: datetime, device_id: Optional[int] = None
    ):
        print(
            f"[LOG] БД: запрос истории с {start} по {end} для устройства {device_id} "
            f"if device_id else 'всех'"
        )

        return []

    def _apply_control(self, request: Request) -> None:
        cmd = request.args.get("database_command", "").strip()
        if cmd:
            print(f"[LOG] БД: принята управляющая команда {cmd!r}")

    def connect(self, request: Optional[Request] = None) -> Dict[str, Union[str, int]]:
        if request is not None:
            self._apply_control(request)
        data = {
            "connection": "ok",
            "storage_path": self.storage_path,
            "records_today": self.records_today,
        }
        print(
            f"[LOG] Подключение к БД выполнено, записей за сегодня: {data['records_today']}, путь: {data['storage_path']}"
        )
        return data

```

Листинг 3 — dashboard.html

```

<!DOCTYPE html>
<html lang="ru">

<head>

```

```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Умная теплица</title>
<link rel="stylesheet" href="{ { url_for('static', filename='css/main.css') } }">
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script src="{ { url_for('static', filename='js/index.js') } }" defer></script>
</head>

<body>
  <main class="container">
    <section class="card main">
      <h2>Агроном (оператор)</h2>
      <div class="grid">
        <article class="card">
          <h2>Датчик температуры</h2>
          <input id="temperature_value" type="text" readonly>
        </article>
        <article class="card">
          <h2>Датчик влажности воздуха</h2>
          <input id="humidity_value" type="text" readonly>
        </article>
        <article class="card">
          <h2>Датчик влажности почвы</h2>
          <input id="soil_value" type="text" readonly>
        </article>
        <article class="card">
          <h2>Вентилятор</h2>
          <input id="fan_power" type="text" readonly>
        </article>
        <article class="card">
          <h2>Насос</h2>
          <input id="pump_power" type="text" readonly>
        </article>
        <article class="card">
          <h2>Контроллер</h2>
          <input id="controller_mode" type="text" readonly>
          <input id="controller_alerts" type="text" readonly>
        </article>
        <article class="card">
          <h2>База данных</h2>
          <input id="db_connection" type="text" readonly>
          <input id="db_records" type="text" readonly>
        </article>
      </div>
    </section>

    <section class="card main control-section">
      <h2>Управление оборудованием</h2>
      <div class="grid">
        <article class="card">
          <h2>Датчик температуры</h2>
          <label class="field-label">Питание</label>
          <select id="temperature_power" class="control-input">
            <option value="">—</option>
            <option value="on">Вкл</option>
            <option value="off">Выкл</option>
          </select>
          <label class="field-label">Калибровка, °C</label>
          <input id="temperature_calibrate" class="control-input"
type="text" placeholder="например 23.5">
        </article>
        <article class="card">

```

```

        <h2>Датчик влажности воздуха</h2>
        <label class="field-label">Питание</label>
        <select id="humidity_power" class="control-input">
            <option value="">—</option>
            <option value="on">Вкл</option>
            <option value="off">Выкл</option>
        </select>
        <label class="field-label">Калибровка, %</label>
        <input id="humidity_calibrate" class="control-input" type="text"
placeholder="например 55">
    </article>
    <article class="card">
        <h2>Датчик влажности почвы</h2>
        <label class="field-label">Питание</label>
        <select id="soil_power" class="control-input">
            <option value="">—</option>
            <option value="on">Вкл</option>
            <option value="off">Выкл</option>
        </select>
        <label class="field-label">Калибровка, %</label>
        <input id="soil_calibrate" class="control-input" type="text"
placeholder="например 40">
    </article>
    <article class="card">
        <h2>Вентилятор</h2>
        <label class="field-label">Мощность, Вт</label>
        <input id="fan_power_cmd" class="control-input" type="text"
placeholder="например 90">
        <label class="field-label">Питание</label>
        <select id="fan_power_state" class="control-input">
            <option value="">—</option>
            <option value="on">Вкл</option>
            <option value="off">Выкл</option>
        </select>
        <label class="field-label">Действие (лог)</label>
        <input id="fan_action" class="control-input" type="text"
placeholder="например ускорить">
    </article>
    <article class="card">
        <h2>Насос</h2>
        <label class="field-label">Мощность, Вт</label>
        <input id="pump_power_cmd" class="control-input" type="text"
placeholder="например 60">
        <label class="field-label">Питание</label>
        <select id="pump_power_state" class="control-input">
            <option value="">—</option>
            <option value="on">Вкл</option>
            <option value="off">Выкл</option>
        </select>
        <label class="field-label">Действие (лог)</label>
        <input id="pump_action" class="control-input" type="text"
placeholder="например полив">
    </article>
    <article class="card">
        <h2>Контроллер</h2>
        <label class="field-label">Режим</label>
        <select id="controller_mode_cmd" class="control-input">
            <option value="">—</option>
            <option value="auto">Авто</option>
            <option value="manual">Ручной</option>
        </select>
        <label class="field-label inline-check">

```



```

        <input id="controller_clear_alerts" type="checkbox">
        Сбросить тревоги
    </label>
</article>
<article class="card">
    <h2>База данных</h2>
    <label class="field-label">Команда (лог)</label>
    <input id="database_command" class="control-input" type="text"
placeholder="например checkpoint">
</article>
<div class="controls">
    <div class="actions">
        <button type="button" id="send_commands" class="btn-
primary">Отправить команды</button>
    </div>
    <label class="field-label">Ответ сервера</label>
    <textarea id="command_result" class="control-input mono"
readonly></textarea>
</div>
</div>
</section>
</main>
</body>

</html>

```

Листинг 4 — main.css

```

:root {
    --font-family-base: Arial, sans-serif;

    --color-bg: #f4f8f5;
    --color-text: #182D4A;
    --color-muted: #4f6155;
    --color-surface: #ffffff;
    --color-surface-alt: #e3ecf7;
    --color-border: #b2c6e2;
    --color-accent: #182d4a;

    --radius-sm: 8px;
    --radius-md: 24px;

    --shadow-card: 0 2px 6px rgba(31, 43, 34, 0.08);
}

body {
    margin: 0;
    font-family: var(--font-family-base);
    background: var(--color-bg);
    color: var(--color-text);
    max-height: 100vh;
}

.container {
    display: flex;
    justify-content: center;
    flex-wrap: wrap;
    gap: 16px;
    padding: 24px;
}

.grid {

```

```

display: grid;
grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
gap: 16px;
margin-top: 18px;
}

.card {
  background: var(--color-surface-alt);
  border: 1px dashed var(--color-border);
  border-radius: var(--radius-sm);
  padding: 18px;
  box-shadow: var(--shadow-card);

  &>h2 {
    font-size: 15px;
    margin-top: 0;
    border-bottom: 1px solid var(--color-border);
    padding-bottom: 7px;
  }

  &>input {
    width: 100%;
    padding: 10px;
    border-radius: var(--radius-sm);
    border: 1px solid var(--color-border);
    font-size: 14px;
    background: var(--color-surface);
    box-sizing: border-box;

    &+& {
      margin-top: 8px;
    }
  }

  &.main {
    background: var(--color-surface);
    border: none;
    border-radius: var(--radius-md);
    flex: 1;

    &>h2 {
      font-size: 18px;
      border-radius: var(--radius-md) var(--radius-md) 0 0;
      background-color: var(--color-accent);
      padding: 18px;
      margin: -18px -18px 0;
      margin-bottom: 18px;
      color: var(--color-surface);
    }
  }

  /* &.control-section {} */

  .control-section & {
    background: var(--color-bg);
  }
}

.field-label {
  display: block;
  color: var(--color-muted);
  margin-top: 16px;
}

```

```

        margin-bottom: 4px;
        font-size: 14px;
    }

    .inline-check {
        display: flex;
        align-items: center;
        gap: 8px;
        margin-top: 12px;
    }

    .control-input {
        width: 100%;
        padding: 10px;
        border-radius: var(--radius-sm);
        border: 1px solid var(--color-border);
        font-size: 14px;
        background: var(--color-surface);
        box-sizing: border-box;

        &.mono {
            font-family: ui-monospace, monospace;
            font-size: 12px;
            height: 100%;
            resize: none;
        }
    }

    select {
        cursor: pointer;
    }

    .controls {
        display: flex;
        flex-direction: column;
        grid-row: 1 / span 2;
        grid-column: auto / -1;
        align-self: stretch;
    }

    /* .actions {} */

    .btn-primary {
        padding: 12px 20px;
        font-size: 14px;
        border-radius: var(--radius-sm);
        border: none;
        background: var(--color-accent);
        color: var(--color-surface);
        cursor: pointer;

        &:hover {
            filter: brightness(1.25);
        }
    }

```

Листинг 5 — index.js

```

function getTemperatureData() {
    $.ajax({
        type: "GET",
        url: "/connect/temperature",

```

```

        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            $("#temperature_value").val(response.value + " " + response.unit);
        },
    });
}

function getHumidityData() {
    $.ajax({
        type: "GET",
        url: "/connect/humidity",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            $("#humidity_value").val(response.value + " " + response.unit);
        },
    });
}

function getSoilData() {
    $.ajax({
        type: "GET",
        url: "/connect/soil",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            $("#soil_value").val(response.value + " " + response.unit);
        },
    });
}

function getFanData() {
    $.ajax({
        type: "GET",
        url: "/connect/fan",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            $("#fan_power").val(
                response.power + " Вт; статус: " + response.status
            );
        },
    });
}

function getPumpData() {
    $.ajax({
        type: "GET",
        url: "/connect/pump",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            $("#pump_power").val(
                response.power + " Вт; статус: " + response.status
            );
        },
    });
}

```

```

    });
}

function getControllerData() {
    $.ajax({
        type: "GET",
        url: "/connect/controller",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            $("#controller_mode").val("Режим: " + response.mode);
            $("#controller_alerts").val("Тревоги: " + response.alerts_count);
        },
    });
}

function getDatabaseData() {
    $.ajax({
        type: "GET",
        url: "/connect/database",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            $("#db_connection").val("Соединение: " + response.connection);
            $("#db_records").val("Записей за сегодня: " + response.records_today);
        },
    });
}

function updateDashboard() {
    getTemperatureData();
    getHumidityData();
    getSoilData();
    getFanData();
    getPumpData();
    getControllerData();
    getDatabaseData();
}

function collectCommandPayload() {
    return {
        temperature_power: $("#temperature_power").val(),
        temperature_calibrate: $("#temperature_calibrate").val(),
        humidity_power: $("#humidity_power").val(),
        humidity_calibrate: $("#humidity_calibrate").val(),
        soil_power: $("#soil_power").val(),
        soil_calibrate: $("#soil_calibrate").val(),
        fan_power_cmd: $("#fan_power_cmd").val(),
        fan_power_state: $("#fan_power_state").val(),
        fan_action: $("#fan_action").val(),
        pump_power_cmd: $("#pump_power_cmd").val(),
        pump_power_state: $("#pump_power_state").val(),
        pump_action: $("#pump_action").val(),
        controller_mode: $("#controller_mode_cmd").val(),
        controller_clear_alerts: ($("#controller_clear_alerts").is(":checked")
            ? "1"
            : ""),
        database_command: $("#database_command").val(),
    };
}

```

```

function sendCommands() {
    $.ajax({
        type: "GET",
        url: "/connect/command",
        dataType: "json",
        contentType: "application/json",
        data: collectCommandPayload(),
        success: function (response) {
            $("#command_result").val(JSON.stringify(response, null, 2));
        },
    });
}

$("#send_commands").on("click", sendCommands);

setInterval(updateDashboard, 1000);
updateDashboard();

```